

双周技术分享之C++宏编程

宏的作用

</>

Plain Text | 收起 ^

```
1 # 定义常量
2 #define N 100
3
4 # 防止多重包含
5 #ifndef XXXX
6 #include XXX
7 #define XXXX
8
9 # 条件编译
10 #ifdef PADDLE_WITH_CUSTOM_DEVICE
11 XXXXXXXX
12 #endif
13
14 # 宏函数
15 #define MAX(a, b) ((a) > (b) ? (a) : (b))
```

宏编程的调试

- `gcc -E` 让编译器在预处理结束后停止，不进行编译、链接
- `gcc -P` 屏蔽编译器输出预处理结果的行标记，减少干扰
- 屏蔽无关的头文件
- 临时删掉不影响宏展开的 `#include` 行

宏编程的特性

因为宏编程本质上只是在预处理阶段的文本替换，输入和输出都是符号，因此没有类型的概念，也不涉及编译时的 C++ 语法。宏编程具有以下特性：

- 一个宏参数是一个任意的符号序列，不同宏参数之间用逗号分隔，额外用括号包住的内容视为一个参数（例如 `FOO(bool, std::pair<int, int>)` 被认为是 `FOO()` 有三个参数：`bool` | `std::pair<int` | `int>`）
- 每个参数可以是空序列，且空白字符会被忽略（例如 `a + 1` 和 `a+1` 相同）
- 在一个参数内，不能出现不配对的括号

针对上面的问题，有两种解决思路：

</>

Plain Text | 收起 ^

```
1 # 1.使用类型别名
2 using IntPair = std::pair<int, int>
3 FOO(bool, IntPair)
4
5 # 2.使用括号对封装每个参数，并在最终展开时移除括号
6 #define REMOVE_PARENS(T) REMOVE_PARENS_IMPL T
7 #define REMOVE_PARENS_IMPL(...) __VA_ARGS__
8
9 #define FOO(A, B) int foo(A x, B y)
10 #define BAR(A, B) FOO(REMOVE_PARENS(A), REMOVE_PARENS(B))
11
12 FOO(bool, IntPair) // -> int foo(bool x, IntPair y)
13 BAR((bool), (std::pair<int, int>)) // -> int foo(bool x, std::pair<int, int> y)
```

符号拼接

在宏编程中，通过拼接标识符 `##` 将宏函数的参数拼接成其他符号，再进一步展开为目标结果，是宏编程的实现基础。符号拼接的过程如下：

1. 在进入宏函数前，所有宏参数会先进行一次预扫描，完全展开未用于拼接标识符或获取字面量的所有参数
2. 在宏函数展开时，用（预扫描展开后的）参数替换展开目标里的同名符号
3. 在宏函数展开后，替换后的文本会进行二次扫描，继续展开结果里出现的宏(相同的宏不会继续展开)

直接拼接示例：

</>

Plain Text | 收起 ^

```
1 #define Foo() foo
2 #define concat(x1, x2) x1##x2
3
4 concat(bar, foo) // ->barfoo
5 concat(bar, Foo()) // ->barFoo()
```

为了使用于拼接的参数也能充分展开，可以使用延迟拼接的操作：

</>

Plain Text | 收起 ^

```
1 # paddle中的延迟拼接, 代码所在路径: paddle/phi/core/macros.h
2 #define PD_CONCATENATE(arg1, arg2) PD_CONCATENATE1(arg1, arg2)
3 #define PD_CONCATENATE1(arg1, arg2) PD_CONCATENATE2(arg1, arg2)
4 #define PD_CONCATENATE2(arg1, arg2) arg1##arg2
```

在预扫描前后, 宏函数都要求参数个数必须匹配, 否则无法展开

</>

Plain Text | 收起 ^

```
1 #define comma() ,
2 PD_CONCATENATE(x comma() y)      //error: too few arguments provided to function-
   like macro invocation
3 PD_CONCATENATE(x, comma())        //error: too many arguments provided to function-
   like macro invocation
```

符号拼接的应用

自增自减

</>

Plain Text | 收起 ^

```
1 #define INC(N) PD_CONCATENATE(INT_, N)
2 #define INC_0 1
3 #define INC_1 2
4 ....
5
6 #define DEC(N) PD_CONCATENATE(DEC_, N)
7 #define DEC_1 0
8 #define DEC_2 1
9 ...
```

布尔转换

</>

Plain Text | 收起 ^

```
1 #define BOOL(N) PD_CONCATENATE(BOOL_, N)
2 #define BOOL_0 0
3 #define BOOL_1 1
4 #define BOOL_2 1
5 ...
```

逻辑运算

</>

Plain Text | 收起 ^

```
1 #define NOT(N) PD_CONCATENATE(NOT_, BOOL(N))
2 #define NOT_0 1
3 #define NOT_1 0
4
5
6 #define AND(A, B) PD_CONCATENATE(AND_, PD_CONCATENATE(BOOL(A), BOOL(B)))
7 #define AND_00 0
8 #define AND_01 0
9 #define AND_10 0
10 #define AND_11 1
11
12 #define OR(A, B) PD_CONCATENATE(OR_, PD_CONCATENATE(BOOL(A), BOOL(B)))
13 #define OR_00 0
14 #define OR_01 1
15 #define OR_10 1
16 #define OR_11 1
```

条件选择

</>

Plain Text | 收起 ^

```
1 #define IF(PRED, THEN, ELSE) PD_CONCATENATE(IF_, BOOL(PRED))(THEN, ELSE)
2 #define IF_0(THEN, ELSE) ELSE
3 #define IF_1(THEN, ELSE) THEN
```

惰性求值

预扫描阶段宏参数展开后可能出现参数个数不匹配的错误，此时可以采用惰性求值，即先返回宏函数名，再传递参数的延迟调用。

</>

Plain Text | 收起 ^

```
1 # 示例
2 IF(1, comma(), 2)    // IF(1,,2) 4个参数,error: too many arguments provided to
    function-like macro invocation
3 IF(1, comma, 2)()    // IF_1(comma, 2) -> comma() -> ,
```

变长参数

从 C++ 11 开始，宏函数支持了变长参数 ...，接受任意个宏参数（用逗号分隔）

- 传入的变长参数可以用 `__VA_ARGS__` 获取
- 允许传递空参数，即 `__VA_ARGS__` 替换为空

对于空参数，展开时需要处理多余逗号的问题：

</>

Plain Text | 收起 ^

```
1 #define log(format, ...) printf("LOG: " format, __VA_ARGS__)
2
3 log("%d%f", 1, .2);    // -> printf("LOG: %d%f", 1, .2);
4 log("hello world");    // -> printf("LOG: hello world", );
5 log("hello world", );  // -> printf("LOG: hello world", );
```

后两种调用 分别对应不传变长参数、变长参数为空的情况，展开结果会 多出一个逗号，导致 C/C++ 编译错误。

为了解决这个问题，一些编译器（例如 gcc/clang）扩展了 `## __VA_ARGS__` 的用法 —— 如果 不传变长参数，则省略前面的逗号：

</>

Plain Text | 收起 ^

```
1 #define log(format, ...) printf("LOG: " format, ## __VA_ARGS__)
2
3 log("%d%f", 1, .2);    // -> printf("LOG: %d%f", 1, .2);
4 log("hello world");    // -> printf("LOG: hello world");
5 log("hello world", );  // -> printf("LOG: hello world", );
```

但传入空参数仍会出现逗号。C++ 20 引入了 `__VA_OPT__` 标识符解决了该问题，如果变长参数是空参数，不展开该符号（不仅限于逗号）：

</>

Plain Text | 收起 ^

```
1 #define log(format, ...) printf("LOG: " format __VA_OPT__(,) __VA_ARGS__)
2
3 log("%d%f", 1, .2);    // -> printf("LOG: %d%f", 1, .2);
4 log("hello world");    // -> printf("LOG: hello world");
5 log("hello world", );  // -> printf("LOG: hello world");
```

变长参数的应用

下标访问

</>

Plain Text | 收起 ^

```
1 #define ARGSELEM(N, ...) PD_CONCATENATE(ARGSELEM_, N)(__VA_ARGS__)
2 #define ARGSELEM_0(a0, ...) a0
```

```
3 #define ARGS_ELEM_1(a0, a1, ...) a1
4 #define ARGS_ELEM_2(a0, a1, a2, ...) a2
5 #define ARGS_ELEM_3(a0, a1, a2, a3, ...) a3
6 ...
7 #define ARGS_ELEM_7(a0, a1, a2, a3, a4, a5, a6, a7, ...) a7
8 #define ARGS_ELEM_8(a0, a1, a2, a3, a4, a5, a6, a7, a8, ...) a8
```

长度计算

</>

Plain Text | 收起 ^

```
1 # paddle中的获取注册kernel的数据类型个数，代码所在路径：paddle/phi/core/kernel_registry.h
2 #define PD_NARGS(...) _PD_NARGS((__VA_ARGS__, _PD_RESQ_N())) //
   __PD_ARG_N_EXPAND(arg1, arg2, agr3, 15, 14, 13, 12, ..., 0) 15:N+1 N:16
3 #define _PD_NARGS(...) _PD_ARG_N(__VA_ARGS__)
4 #define _PD_ARG_N_EXPAND(                                     \
5     _1, _2, _3, _4, _5, _6, _7, _8, _9, _10, _11, _12, _13, _14, _15, N, ...) \
6     N
7 #define _PD_ARG_N(args) _PD_ARG_N_EXPAND args
8 #define _PD_RESQ_N() 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0
```

遍历访问

</>

Plain Text | 收起 ^

```
1 # paddle实例化不同数据类型kernel的代码，代码所在路径：paddle/phi/core/kernel_registry.h
2 #define _PD_KERNEL_INSTANTIATION(                                     \
3     N, meta_kernel_fn, backend, context, kernel_instantiation_macro, ...) \
4     PD_CONCATENATE(_PD_KERNEL_INSTANTIATION_, N) \
5     (meta_kernel_fn, backend, context, kernel_instantiation_macro, __VA_ARGS__)
6
7 #define _PD_KERNEL_INSTANTIATION_1(                                     \
8     meta_kernel_fn, backend, context, kernel_instantiation_macro, cpp_dtype) \
9     kernel_instantiation_macro(meta_kernel_fn, cpp_dtype, context)
10 #define _PD_KERNEL_INSTANTIATION_2(meta_kernel_fn, \
11                                     backend, \
12                                     context, \
13                                     kernel_instantiation_macro, \
14                                     cpp_dtype, \
15                                     ...) \
16     kernel_instantiation_macro(meta_kernel_fn, cpp_dtype, context) \
17     PD_EXPAND(_PD_KERNEL_INSTANTIATION_1(meta_kernel_fn, \
18                                     backend, \
```

```
19             context, \
20             kernel_instantiation_macro, \
21             __VA_ARGS__))
22 #define _PD_KERNEL_INSTANTIATION_3(meta_kernel_fn, \
23             backend, \
24             context, \
25             kernel_instantiation_macro, \
26             cpp_dtype, \
27             ...) \
28     kernel_instantiation_macro(meta_kernel_fn, cpp_dtype, context) \
29     PD_EXPAND(_PD_KERNEL_INSTANTIATION_2(meta_kernel_fn, \
30             backend, \
31             context, \
32             kernel_instantiation_macro, \
33             __VA_ARGS__))
```